

TheKnife

Manuale Tecnico

Sviluppato da: Gianluca Melis,
Alessandro Melnyk,
Davide Redemagni,
Simone Zamberletti

Sommario

1. Introduzione al Progetto	2
2. Struttura Tecnica dell'Applicazione	3
3. Progettazione del Software e Modellazione UML	6
4. Strutture Dati e Trasferimento Informazioni	13
5. Scelte di Progettazione Architetture e Design Pattern	16
6. Scelte Algoritmiche e Ottimizzazione	18
7. Progettazione e Struttura della Base di Dati (PostgreSQL)	20

1.Introduzione al Progetto

1.1 Obiettivi e Contesto Applicativo

La piattaforma software TheKnife nasce con l'obiettivo di fornire un sistema completo e centralizzato per la ricerca, la consultazione e la gestione di esercizi di ristorazione distribuiti a livello globale. Ispirandosi ai principali servizi di settore, l'applicazione consente agli utenti finali di individuare locali in base a molteplici parametri di filtraggio (località geografica, coordinate, tipologia di cucina, fasce di prezzo e servizi offerti come delivery o prenotazione online), consultare e pubblicare recensioni corredate da valutazioni numeriche, nonché gestire un elenco personalizzato di ristoranti preferiti. Contestualmente, la piattaforma offre agli utenti con profilo di ristoratore un'area dedicata per l'inserimento dei propri locali, il monitoraggio delle statistiche di gradimento e la gestione delle risposte ai feedback ricevuti dai clienti.

1.2 Architettura Distribuita a 3 Livelli (3-Tier)

Dal punto di vista ingegneristico, il sistema abbandona qualsiasi meccanismo di memorizzazione locale isolata su file per adottare una solida architettura distribuita a tre livelli (3-Tier Architecture), progettata per garantire scalabilità, concorrenza e separazione dei compiti:

- **Livello di Presentazione** (Client Tier - ClientTK): Realizzato in ambiente Java con interfaccia grafica interattiva basata su framework JavaFX, consente l'interazione per tutte le tipologie di utenza (ospiti, clienti registrati e ristoratori). Tutte le operazioni di rete sono gestite in modo asincrono su thread worker dedicati, preservando la reattività della GUI.
- **Livello Applicativo e di Business Logic** (Server Tier - ServerTK): Costituisce il nucleo di elaborazione del back-end, implementato tramite tecnologia Java RMI (Remote Method Invocation). Il server accoglie dinamicamente le credenziali di accesso al DBMS all'avvio, gestisce il pool di thread per le richieste concorrenti dei client ed elabora le regole di business tramite interfacce DAO.
- **Livello di Persistenza dei Dati** (Database Tier - dbTK): Affidato al DBMS relazionale PostgreSQL, garantisce la memorizzazione centralizzata, l'integrità referenziale, la consistenza transazionale (proprietà ACID) e l'ottimizzazione delle query di ricerca avanzata.

2. Struttura Tecnica dell'Applicazione

2.1 Organizzazione dei Package e Moduli

L'infrastruttura del codice sorgente è organizzata all'interno del namespace radice theknife, strutturato in sottopackage specializzati per garantire una rigida separazione dei livelli architetturali e favorire la manutenibilità del sistema:

- **theknife:** contiene le classi del modello di dominio, le strutture dati comuni, le classi di supporto all'esecuzione e tutte le viste dell'interfaccia grafica JavaFX (MainApp, LoginView, GuestSearchView, ClientDashboardView, RestaurateurPanelView, ecc.).
- **theknife.client:** racchiude il gestore della connessione di rete remota (RmiClientManager), implementato con pattern Singleton per centralizzare il lookup e l'accesso allo stub RMI del servizio.
- **theknife.server:** ospita il punto di ingresso del processo di back-end (ServerTK), deputato all'acquisizione interattiva delle credenziali, alla configurazione della connessione a PostgreSQL e all'avvio del Registry RMI.
- **theknife.remote:** definisce l'interfaccia remota RMI (TheKnifeService) e la sua implementazione concreta (TheKnifeServiceImpl), che estende UnicastRemoteObject per smistare le chiamate dei client verso il layer dei DAO.
- **theknife.dao:** incapsula il layer di accesso ai dati, definendo i contratti (UtenteDAO, RistoranteDAO, RecensioneDAO) e le rispettive implementazioni JDBC (UtenteDAOImpl, RistoranteDAOImpl, RecensioneDAOImpl) per l'interazione con il database relazionale.
- **theknife.db:** contiene la classe di gestione della connettività (DatabaseConnection), responsabile dell'allocazione dinamica, configurabile e thread-safe delle connessioni JDBC verso PostgreSQL.

2.2 Classi di Dominio

Le entità fondamentali che modellano il dominio applicativo implementano l'interfaccia `java.io.Serializable` per consentire il marshaling e la trasmissione attraverso il middleware RMI:

- **Ristorante:** modella le proprietà di un esercizio di ristorazione, inclusi identificativo univoco, denominazione, indirizzo, città, nazione, coordinate geografiche di precisione (latitudine e longitudine), fascia di prezzo medio, disponibilità di consegna o prenotazione online e lista delle tipologie di cucina associate.
- **Utente:** classe base astratta che astrae gli attributi anagrafici comuni (nome, cognome, username, password cifrata, dataNasc, domicilio, ruolo), integrando metodi di validazione sintattica.
- **UtenteRegistrato:** specializzazione di `Utente` per i clienti della piattaforma; include i metodi per interagire con il servizio remoto per l'aggiunta, la modifica e l'eliminazione di recensioni e la gestione dell'elenco dei preferiti.
- **Ristoratore:** estensione di `Utente` dedicata ai gestori; fornisce i metodi per la creazione di nuovi ristoranti, l'interrogazione dei locali posseduti e la pubblicazione di risposte ai feedback dei clienti.
- **UtenteNonRegistrato:** modella la sessione di consultazione anonima (Guest), consentendo l'avvio della ricerca e la procedura di registrazione.
- **Recensione:** rappresenta sia la valutazione primaria rilasciata da un cliente (con voto compreso tra 1 e 5, commento e data), sia l'eventuale risposta inviata dal ristoratore (collegata tramite il campo `idRecensionePadre`).
- **TipoCucina, Ruolo, CriterioRicerca:** enumerazioni che garantiscono la sicurezza dei tipi per le categorie culinarie, i privilegi di accesso e i parametri di filtraggio.

2.3 Layer di Servizio e Punti di Ingresso

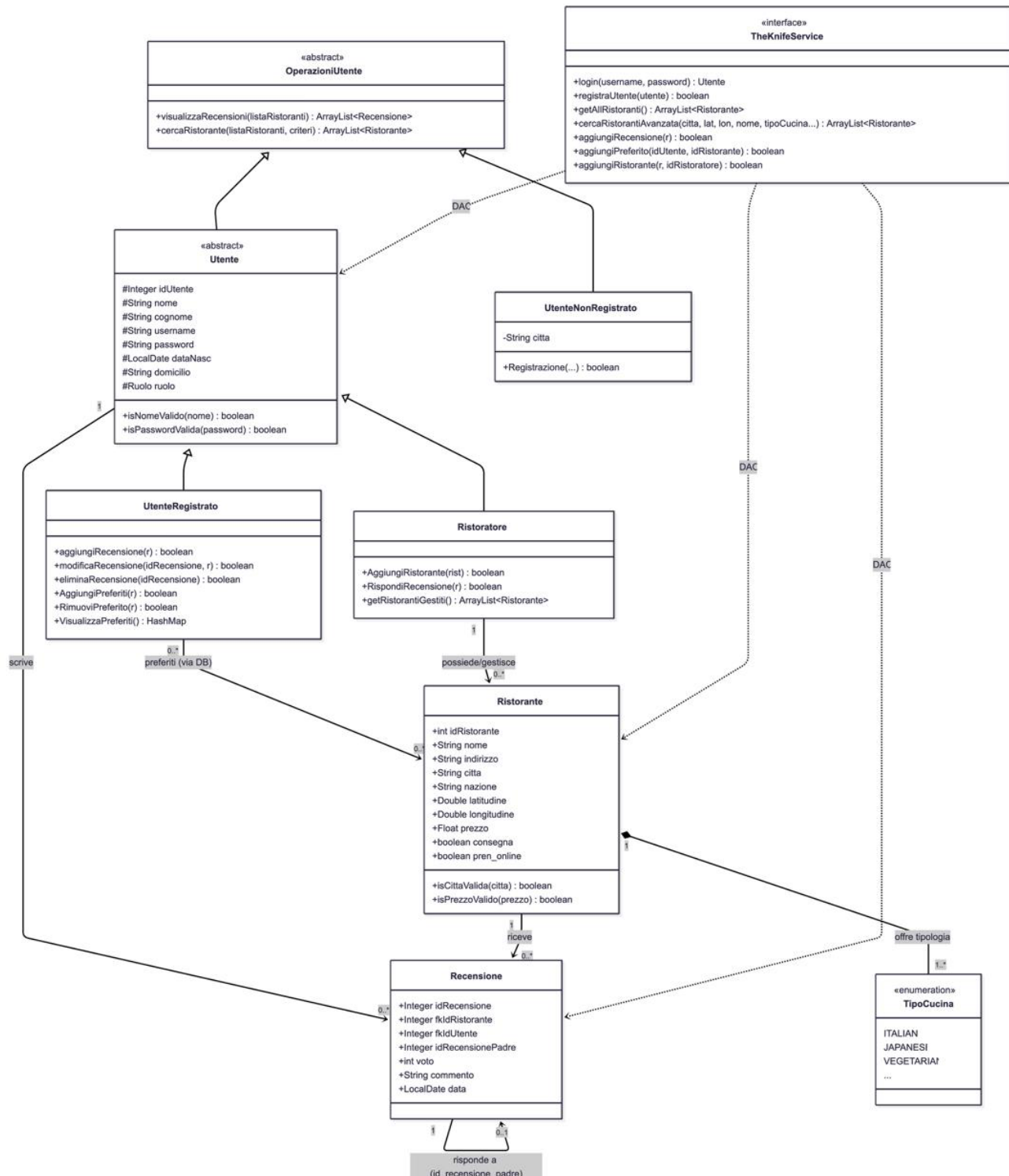
Il sistema si articola su due eseguibili autonomi generati come artefatti .jar distinti dal build di Maven:

- **ServerTK (Eseguibile Server):** classe principale del back-end; richiede a riga di comando l'host, la porta, il nome del database e le credenziali di PostgreSQL, verifica la raggiungibilità del DBMS, istanzia il Registry RMI sulla porta 1099 ed esporta il servizio TheKnifeService rimanendo in attesa perpetua di connessioni dai client.
- **TheKnife / MainApp (Eseguibile Client):** classe di avvio del front-end grafico JavaFX; all'avvio tenta il collegamento con il Registry RMI tramite RmiClientManager e inizializza la navigazione attraverso le diverse viste grafiche per ospiti, clienti e ristoratori.

3. Progettazione del Software e Modellazione UML

3.1 Struttura Statica del Sistema: Diagramma delle Classi (Class Diagram)

La struttura statica del sistema modella le entità di dominio, i contratti per le operazioni utente e l'interfaccia remota del middleware distribuito.



L'architettura statica si articola attorno ai seguenti principi di modellazione ad oggetti:

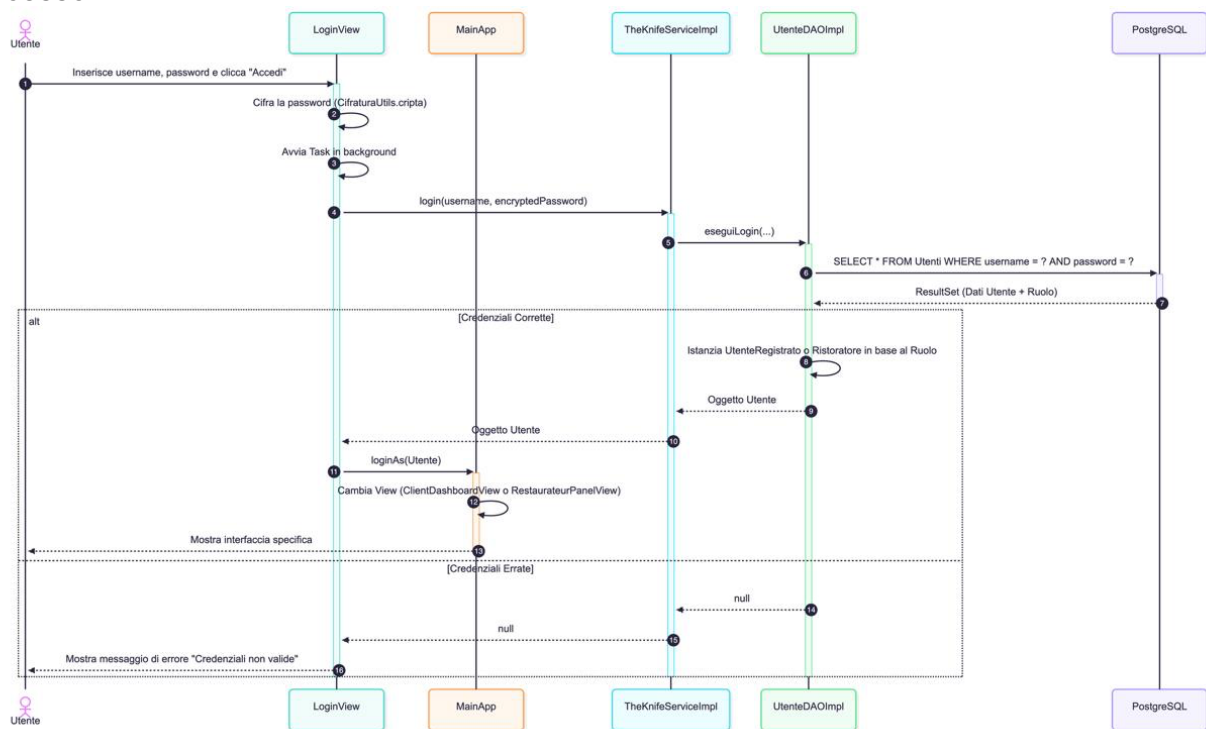
- **Gerarchia e Polimorfismo delle Entità Utente:** La classe astratta Utente modella lo stato anagrafico comune (idUserente, nome, cognome, username, password, dataNasc, domicilio, ruolo) ed estende OperazioniUtente. Da essa derivano due sottoclassi concrete:
- **UtenteRegistrato:** implementa i privilegi del cliente finale, integrando la gestione remota delle recensioni (aggiungiRecensione, modificaRecensione, eliminaRecensione) e la sincronizzazione della lista dei preferiti (AggiungiPreferiti, RimuoviPreferito, VisualizzaPreferiti).
- **Ristoratore:** specializza Utente per i gestori, fornendo i metodi per la registrazione di nuove strutture (AggiungiRistorante), la risposta ai feedback (RispondiRecensione) e il recupero dei locali amministrati (getRistorantiGestiti).
- **UtenteNonRegistrato:** modella la consultazione da parte dell'utente anonimo (Guest), incapsulando la località temporanea di ricerca e la procedura di registrazione (Registrazione).
- **Modellazione del Ristorante e Tipologie di Cucina:** La classe Ristorante aggrega attributi anagrafici, indicatori logistici (consegna, pren_online), coordinate di precisione (latitudine, longitudine) e una collezione di enum TipoCucina con cardinalità 1..*.
- **Associazione Recensioni e Risposte Ricorsive:** Recensione funge da entità di collegamento tra Utente e Ristorante. La classe modella una relazione ricorsiva 0..1->1 tramite il campo idRecensionePadre: se valorizzato a -1, identifica una recensione primaria scritta da un cliente (con voto 1..5); se contiene l'ID di un'altra recensione, identifica la singola risposta testuale del ristoratore.
- **Disaccoppiamento tramite Interfaccia Remota TheKnifeService:**
L'interfaccia TheKnifeService (estendente java.rmi.Remote) astrae le operazioni di back-end disponibili ai client, disaccoppiando il dominio dall'accesso ai dati persistenti.

3.2 Comportamento Dinamico: Diagrammi di Sequenza (Sequence Diagrams)

Per descrivere la dinamica del sistema e la cooperazione asincrona tra i diversi livelli architetturali (Client JavaFX, Back-end RMI e Database PostgreSQL), sono stati modellati i tre casi d'uso critici.

3.2.1 Processo di Autenticazione e Accesso

Il diagramma di sequenza del login descrive l'interazione per la verifica delle credenziali d'accesso.

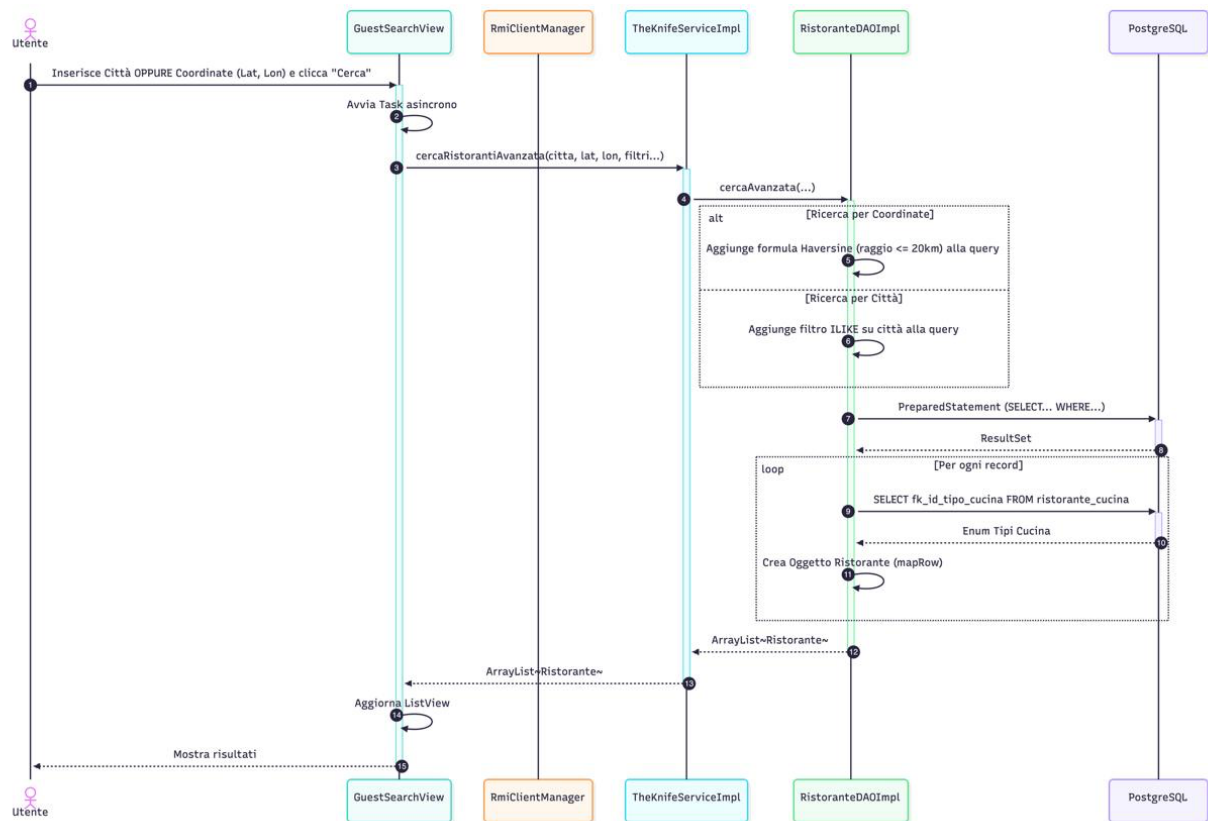


1. L'utente immette username e password nella LoginView e attiva il comando di accesso.
2. La vista invoca `Gestione.CifraturaUtils.cifra()` per cifrare la password tramite algoritmo AES prima della trasmissione sulla rete.
3. Per evitare il blocco del JavaFX Application Thread, viene avviato un `javafx.concurrent.Task` in background.
4. Il client invoca via RMI il metodo `login(username, encryptedPassword)` esposto da `TheKnifeServiceImpl`.
5. `TheKnifeServiceImpl` delega la verifica a `UtenteDAOImpl.eseguiLogin()`, che esegue una query con `PreparedStatement` su `PostgreSQL`.

6. Dal ResultSet ottenuto, UtenteDAOImpl istanzia l'oggetto polimorfo corretto (UtenteRegistrato o Ristoratore in base alla colonna ruolo) e lo restituisce attraverso la catena di chiamata.

7. MainApp riceve l'entità, aggiorna la sessione globale e commuta la scena verso la vista dedicata (LoggedInClientView o LoggedInRestaurateurView). In caso di credenziali errate o nulle, viene mostrato un messaggio di errore.

3.2.2 Ricerca Avanzata e Filtraggio Ristoranti



Il diagramma di sequenza illustra il flusso di esecuzione delle ricerche multi-criterio e di prossimità spaziale.

1. Dalla GuestSearchView, l'utente imposta i parametri di ricerca (città oppure coordinate geografiche, fascia di prezzo, tipologia di cucina, servizi di delivery/prenotazione, rating minimo) e avvia la ricerca.
2. L'operazione viene demandata a un task asincrono che chiama il metodo remoto `cercaRistorantiAvanzata()` tramite `RmiClientManager`.
3. `TheKnifeServiceImpl` inoltra i parametri a `RistoranteDAOImpl.cercaAvanzata()`.

4. Il DAO costruisce dinamicamente la query SQL parametrata su RistorantiTheKnife:

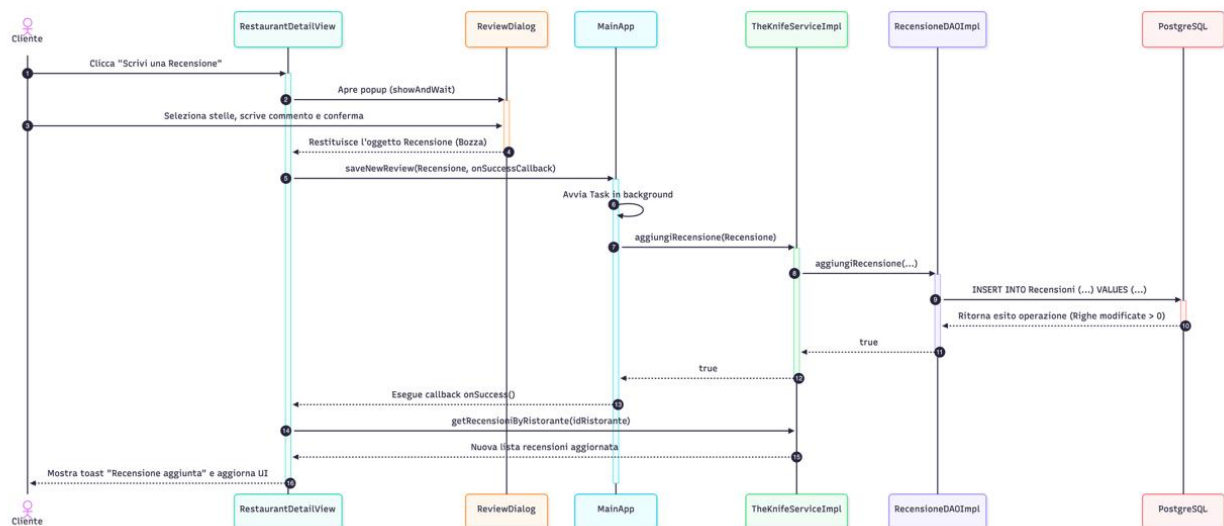
- Se la ricerca include coordinate, calcola la distanza spaziale integrando la formula dell'emisenoverso (Haversine) direttamente nella clausola WHERE per filtrare i locali entro il raggio stabilito.
- Se la ricerca avviene per testo, applica filtri case-insensitive ILIKE / LOWER().
- Esegue la JOIN con la tabella Recensioni per valutare il filtro aggregato HAVING AVG(voto) >= ?.

5. Per ciascun record estratto dal ResultSet, il DAO recupera le tipologie culinarie collegate e istanzia un oggetto Ristorante.

6. La lista risultante (ArrayList<Ristorante>) viene trasmessa al client via RMI e visualizzata nella ListView della GUI senza bloccare il thread principale.

3.2.3 Inserimento e Gestione Recensioni

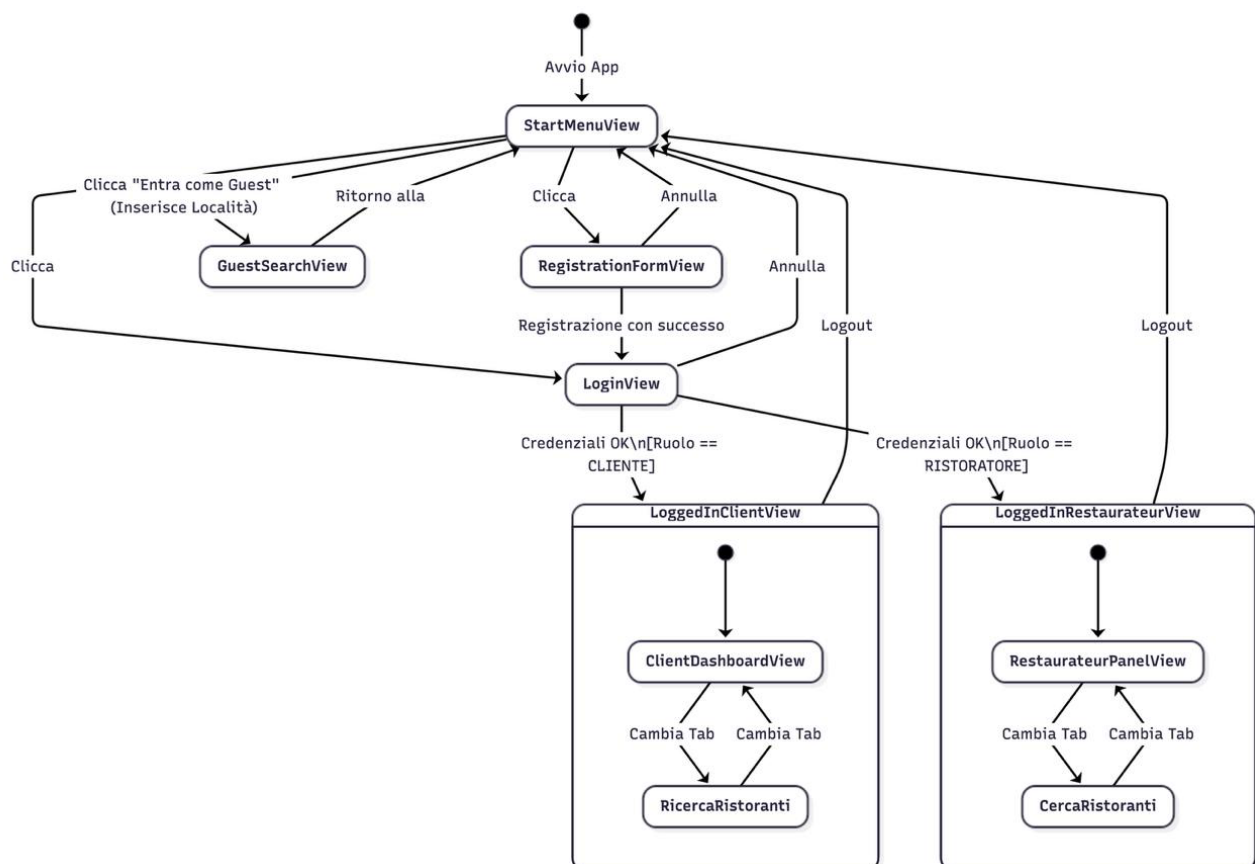
Il diagramma di sequenza descrive la pubblicazione transazionale di una recensione con aggiornamento reattivo dell'interfaccia.



1. All'interno di RestaurantDetailView, un cliente autenticato seleziona l'azione di recensione, aprendo il pop-up modale ReviewDialog.
2. L'utente compila la valutazione tramite star selector (voto da 1 a 5) e inserisce il commento testuale.
3. Alla conferma, ReviewDialog produce un'istanza di Recensione.
4. MainApp esegue in background il salvataggio invocando il metodo remoto aggiungiRecensione() del server.
5. TheKnifeServiceImpl trasferisce l'oggetto a RecensioneDAOImpl.aggiungiRecensione().
6. Il DAO esegue l'inserimento atomico su PostgreSQL. Se si tratta di una risposta inviata da un ristorante (id_recensione_padre != -1), il DAO verifica preventivamente l'assenza di risposte preesistenti per garantire il vincolo di unicità.
7. Al completamento con successo della transazione, viene eseguita la callback onSuccess() che ricarica la lista aggiornata delle recensioni dal server e notifica l'avvenuto salvataggio all'utente tramite toast informativo.

3.3 Ciclo di Vita della Navigazione: Diagramma degli Stati (State Diagram)

Il diagramma degli stati descrive il ciclo di navigazione dell'interfaccia grafica e le transizioni tra le sessioni utente all'interno del client JavaFX.



- **Stato Iniziale (StartMenuView):** Punto d'ingresso all'avvio dell'applicazione. Consente tre diramazioni:

- Accesso Guest: richiede l'immissione di una località e transiziona direttamente alla schermata GuestSearchView in modalità consultazione anonima.
 - Registrazione: commuta lo stato in RegistrationFormView; in caso di successo della creazione account, instrada l'utente verso LoginView.
 - Login: conduce a LoginView per l'inserimento delle credenziali.
-
- **Stato Cliente Autenticato (LoggedInClientView):** Raggiunto in caso di autenticazione con ruolo CLIENTE. La vista gestisce un contenitore TabPane a due schede:
 - Scheda ClientDashboardView: visualizzazione e rimozione dei ristoranti preferiti, gestione ad albero delle proprie recensioni con facoltà di modifica ed eliminazione.
 - Scheda RicercaRistoranti: ricerca avanzata con filtri e possibilità di accedere al dettaglio per aggiungere nuovi locali ai preferiti o rilasciare valutazioni.
-
- **Stato Ristoratore Autenticato (LoggedInRestaurateurView):** Raggiunto in caso di autenticazione con ruolo RISTORATORE. Organizzato anch'esso tramite TabPane:
 - Scheda RestaurateurPanelView: pannello diviso a SplitPane per la consultazione dei ristoranti posseduti, registrazione di nuovi locali tramite form dedicato, monitoraggio delle recensioni ricevute e invio della singola risposta autorizzata.
 - Scheda CercaRistoranti: consultazione generale dei ristoranti presenti sulla piattaforma.
-
- **Transizione di Logout:** Da ciascuno stato autenticato, l'invocazione del logout invalida l'istanza dell'utente corrente e ripristina lo stato iniziale StartMenuView.

4. Strutture Dati e Trasferimento Informazioni

4.1 Collezioni e Data Transfer Objects (DTO)

Nel passaggio dall'architettura monolitica basata su file a quella distribuita client-server, il ruolo delle strutture dati in memoria è stato riadattato: non fungono più da archivio persistente primario, ma da vettori volatili di trasferimento (Data Transfer Objects o DTO) e da strutture di supporto per la manipolazione efficiente dei dati sui singoli nodi.

- **Utilizzo di ArrayList<T>:**

- **Trasferimento dati via RMI:** Le collezioni di tipo ArrayList rappresentano la struttura primaria restituita dai metodi remoti di TheKnifeService (es. getAllRistoranti(), cercaRistorantiAvanzata(), getRecensioniByRistorante()). Essendo serializzabile, ArrayList garantisce un incapsulamento compatto dei record estratti dal DBMS per il loro instradamento sulla rete tramite Java RMI.
- **Accesso indicizzato e binding con JavaFX:** Sul nodo client, i dati ricevuti vengono convertiti in ObservableList (FXCollections.observableArrayList(...)) per alimentare in modo reattivo le componenti grafiche (ListView, TreeView, ComboBox). La struttura ad array ridimensionabile consente un accesso posizionale istantaneo in tempo costante $O(1)$, ottimale per la selezione degli elementi e l'aggiornamento visuale delle celle.

- **Impiego di HashMap<K, V>:**

- La mappa hash viene utilizzata per associazioni chiave-valore in memoria. In particolare, è adottata in ClientDashboardView per raggruppare gerarchicamente le recensioni scritte dall'utente attorno all'ID del rispettivo ristorante (Map<Integer, Treeltem<Object>>).
- L'efficienza temporale media di inserimento e recupero in tempo costante $O(1)$ assicura che l'albero visivo (TreeView) venga popolato senza degradare le prestazioni dell'interfaccia utente.

- **Serializzazione e Conformità RMI:**

- Tutte le classi che compongono il payload di rete (Ristorante, Utente, UtenteRegistrato, Ristoratore, Recensione) implementano tassativamente `java.io.Serializable` e dichiarano esplicitamente il campo `private static final long serialVersionUID`.
- Questa accortezza previene errori di `InvalidClassException` durante la fase di unmarshaling dei byte-stream tra client e server, assicurando l'interoperabilità dei tipi trasmessi.

4.2 Type-Safety tramite Enumerazioni

Per impedire l'introduzione di stati inconsistenti o valori anomali a livello applicativo e sul database, il dominio fa largo uso di tipi enumerativi fortemente tipizzati (enum):

- **Ruolo:** Definisce i profili di autorizzazione previsti dalla piattaforma (CLIENTE, RISTORATORE). L'uso dell'enum vincola le decisioni di routing grafico nell'interfaccia utente e mappa in modo sicuro la clausola CHECK (ruolo IN ('CLIENTE', 'RISTORATORE')) definita nello schema relazionale di PostgreSQL.
- **TipoCucina:** Cataloga in modo esaustivo le categorie gastronomiche internazionali (es. ITALIAN, JAPANESE, MEDITERRANEAN_CUISINE, ecc.). La classe include un metodo statico di normalizzazione `fromString(String input)` che processa le stringhe provenienti da form o dataset esterni (rimuovendo accenti, caratteri speciali e uniformando gli spazi in underscore), garantendo un parsing tollerante ai fallimenti e la conversione corretta verso il tipo enum.
- **CriterioRicerca:** Enumera le chiavi di filtraggio applicabili nelle operazioni di interrogazione (CITTA, NAZIONE, TIPO_CUCINA, NOME, PREZZO), garantendo la leggibilità del codice e la manutenibilità in caso di estensioni future dei criteri.

5. Scelte di Progettazione Architeturale e Design Pattern

5.1 Pattern Architeturale Data Access Object (DAO)

Per garantire una netta separazione tra la logica di business e i meccanismi di persistenza dei dati, il back-end del sistema implementa il design pattern Data Access Object (DAO).

L'architettura definisce contratti astratti tramite interfacce Java

(UtenteDAO, RistoranteDAO, RecensioneDAO) che dichiarano le operazioni CRUD necessarie al funzionamento dell'applicazione. Le rispettive classi di implementazione concrete (UtenteDAOImpl, RistoranteDAOImpl, RecensioneDAOImpl) incapsulano l'intero codice SQL, la gestione delle transazioni JDBC e il mapping dei record estratti (ResultSet) verso gli oggetti di dominio. Questa astrazione garantisce che l'implementazione del servizio remoto (TheKnifeServiceImpl) rimanga completamente agnostica rispetto alla tecnologia di persistenza sottostante: qualsiasi modifica allo schema del database relazionale o alle query SQL rimane confinata all'interno del package theknife.dao, preservando l'integrità del layer applicativo e dei client.

5.2 Middleware Distribuito Java RMI (Remote Method Invocation) a 3 Livelli

La transizione a un'architettura client-server distribuita a tre livelli è stata realizzata sfruttando la tecnologia nativa **Java RMI**:

- **Definizione dell'Interfaccia Remota:**
L'interfaccia TheKnifeService estende java.rmi.Remote e dichiara tutti i metodi accessibili dall'esterno, ciascuno corredato dalla clausola throws RemoteException per la gestione trasparente degli errori di rete.
- **Esportazione e Skeleton:**
La classe TheKnifeServiceImpl estende UnicastRemoteObject, consentendo al runtime Java di istanziare lo scheletro di ricezione e instradare le invocazioni remote in ingresso verso i DAO.
- **Naming e Registry:** La classe di avvio ServerTK avvia o si aggancia al Registry RMI sulla porta standard 1099 e vi registra l'oggetto remoto tramite l'operazione rebind("TheKnifeService", service).
- **Stub Remoto:** I client interrogano il registro tramite la classe RmiClientManager per ottenere un riferimento locale (stub) con cui invocare i metodi come se risiedessero nella stessa JVM, astraendo interamente la complessità della serializzazione su socket TCP.

5.3 Gestione della Concorrenza Multi-Client, Thread-Safety e Transazionalità

L'erogazione parallela dei servizi a più utenti connessi da postazioni differenti è garantita attraverso una gestione della concorrenza strutturata su tutti i livelli del sistema:

- **Thread-Safety del Back-end (Server Stateless):** Il runtime RMI gestisce le richieste remote concorrenti assegnando ciascuna chiamata a un thread worker prelevato da un ThreadPool interno alla JVM. Per prevenire corruzioni di memoria e race conditions, TheKnifeServiceImpl e le classi DAO sono rigorosamente stateless: non mantengono stati di sessione o collezioni mutabili condivise.
- **Isolamento delle Connessioni JDBC:** La classe DatabaseConnection evita l'anti-pattern del singleton con connessione condivisa; il metodo getConnection() alloca e restituisce una connessione PostgreSQL distinta e isolata per ciascun thread, rilasciata immediatamente al termine dell'operazione tramite blocchi try-with-resources.
- **Gestione delle Transazioni ACID:** Nelle operazioni atomiche complesse (ad esempio, l'eliminazione a cascata di una recensione e delle sue risposte in RecensioneDAOImpl.eliminaRecensione), l'autocommit viene disabilitato (conn.setAutoCommit(false)), eseguendo il conn.commit() solo al termine di tutte le query o invocando conn.rollback() in caso di SQLException.
- **Concorrenza Asincrona sul Client (JavaFX Application Thread):** Per preservare la fluidità e la reattività della GUI evitando congelamenti durante le chiamate di rete o le elaborazioni pesanti, tutte le interazioni RMI lato client vengono delegate a task asincroni (javaafx.concurrent.Task<T>) eseguiti su thread dedicati (new Thread(task).start()), gestendo l'aggiornamento dei nodi grafici esclusivamente all'interno dei listener setOnSucceeded e setOnFailed.

5.4 Pattern Creazionali, Strutturali e Cifratura AES

- **Pattern Singleton (RmiClientManager):** Implementato sul client con costruttore privato e metodo sincronizzato getInstance(), assicura che esista una sola istanza del manager di connessione e un unico stub RMI condiviso per l'intera durata dell'applicazione.
- **Pattern Helper / Utility Class (Gestione.CifraturaUtils):** La classe di utilità statica incapsula l'algoritmo simmetrico AES (Advanced Encryption Standard) a 128 bit. I metodi cripta(String) e decripta(String) gestiscono la cifratura delle password degli utenti prima dell'inoltro sulla rete RMI e la loro codifica/decodifica in formato Base64 per la memorizzazione protetta sul database PostgreSQL.

6. Scelte Algoritmiche e Ottimizzazione

6.1 Algoritmo di Ordinamento Merge Sort

Per l'ordinamento in memoria di collezioni di oggetti Ristorante (utilizzato per presentare liste ordinate in base a criteri specifici sul client o per elaborazioni interne), il sistema adotta l'algoritmo Merge Sort, implementato nella classe di utilità Gestione.Sort.

- **Analisi della Complessità:** A differenza di algoritmi elementari (come Bubble Sort o Insertion Sort) o di algoritmi con caso peggiore quadratico come il Quick Sort ($O(n^2)$), il Merge Sort garantisce una complessità temporale asintotica deterministica di $\Theta(n \log n)$ sia nel caso medio che nel caso peggiore. Questa stabilità garantisce tempi di risposta costanti e prevedibili durante la manipolazione delle liste nella GUI.
- **Implementazione e Parametrizzazione:** L'algoritmo è strutturato secondo il paradigma Divide et Impera di tipo ricorsivo. Riceve in ingresso un ArrayList<Ristorante> e un'istanza di Comparator<Ristorante>, consentendo di variare dinamicamente la logica di confronto (per nome alfabetico, fascia di prezzo crescente/decrecente o valutazione media) senza modificare la struttura dell'algoritmo.
- **Stabilità dell'Ordinamento:** Il Merge Sort preserva l'ordine relativo degli elementi con chiavi equivalenti (stabilità), una proprietà desiderabile quando si applicano ordinamenti successivi su viste a tabella o liste JavaFX.

6.2 Motore di Ricerca Spaziale (Formula di Haversine) e Ottimizzazione delle Query SQL

Nel nuovo impianto architetturale basato su PostgreSQL, la logica di ricerca non scansa più sequenzialmente collezioni in memoria con complessità lineari $O(n \cdot m)$, ma sfrutta l'elaborazione ad alte prestazioni del motore relazionale del DBMS attraverso query dinamiche ottimizzate:

- **Costruzione Dinamica della Query Parametrizzata:** All'interno di `RistoranteDAOImpl.cercaAvanzata()`, la clausola SQL WHERE viene assemblata tramite `StringBuilder` concatenando esclusivamente i predicati corrispondenti ai filtri effettivamente valorizzati dall'utente (città, nome, tipo cucina, prezzo minimo/massimo, delivery, prenotazione online). L'uso dei `PreparedStatement` garantisce la pre-compilazione del piano di esecuzione e azzerare il rischio di vulnerabilità da SQL Injection.
- **Calcolo della Prossimità Spaziale (Formula dell'Emisfero - Haversine):**
Per soddisfare il requisito di ricerca dei ristoranti vicini alla posizione geografica del guest o al domicilio dell'utente registrato, la distanza superficiale geodetica d tra le coordinate utente (lat_1, lon_1) e le coordinate del ristorante (lat_2, lon_2) viene calcolata tramite la formula di Haversine integrata direttamente nella query SQL:
- **Differenza di Latitudine e Longitudine (in radianti):**
 - $\Delta lat = lat_2 - lat_1$
 - $\Delta lon = lon_2 - lon_1$
- **Calcolo del termine angolare (a):**
 - $a = \sin^2(\Delta lat / 2) + \cos(lat_1) \cdot \cos(lat_2) \cdot \sin^2(\Delta lon / 2)$
- **Calcolo della distanza superficiale (d):**
 - $d = 2 \cdot R \cdot \arcsin(\sqrt{a})$

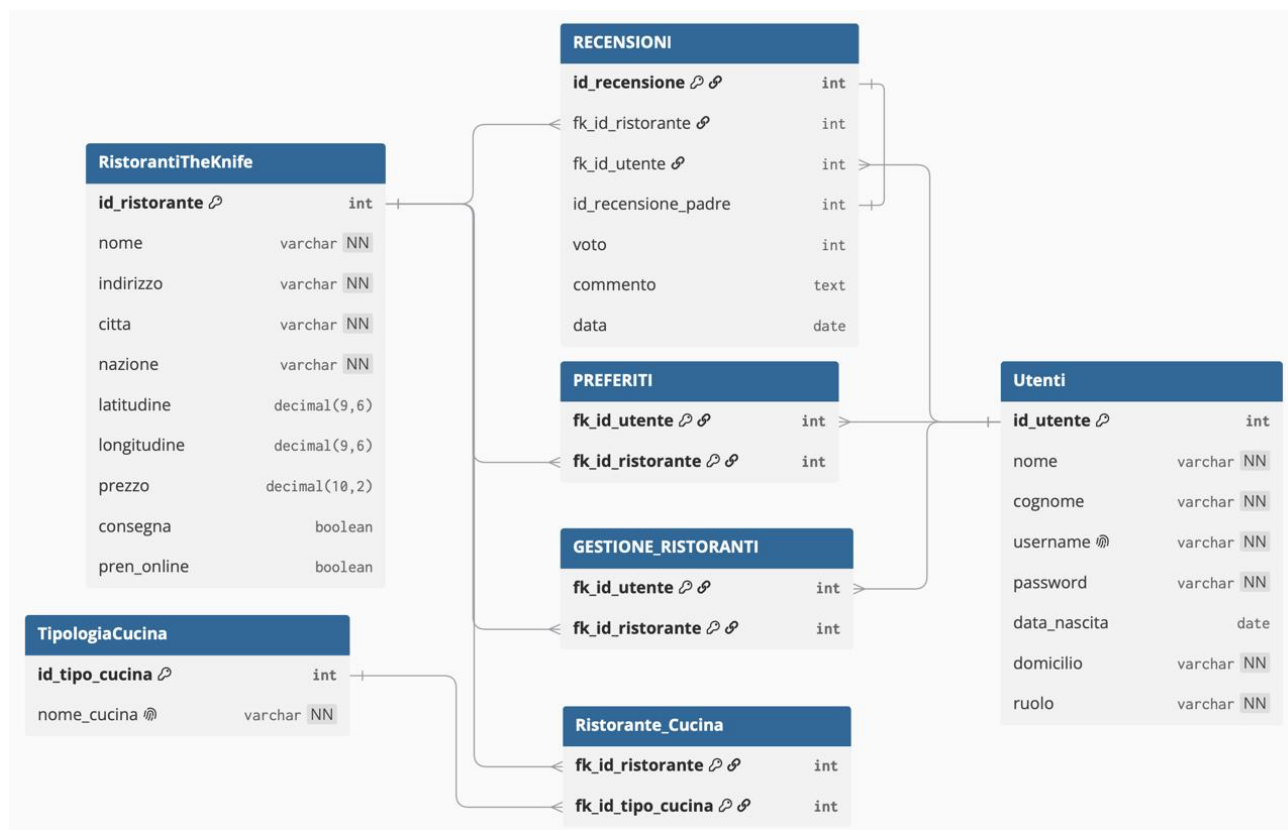
(dove $R \sim 6371km$ è il raggio medio terrestre). Questo calcolo permette di filtrare i record a livello database entro un raggio prefissato (es. $d \leq 20km$) ordinando i risultati per distanza crescente con un impatto computazionale trascurabile.

- **Aggregazione e Clausola HAVING:** Il filtraggio per valutazione media minima (`ratingMin`) viene delegato all'aggregazione relazionale `LEFT JOIN Recensioni ... GROUP BY r.id_ristorante HAVING COALESCE(AVG(rec.voto), 0) >= ?`, evitando il trasferimento non necessario di tuple non conformi dal DBMS verso il server applicativo.

7. Progettazione e Struttura della Base di Dati (PostgreSQL)

7.1 Modello Concettuale e Schema Relazionale

La memorizzazione e la gestione persistente dei dati del sistema TheKnife sono affidate al DBMS relazionale PostgreSQL (dbTK). Lo schema relazionale è stato progettato e normalizzato in Terza Forma Normale (3NF) per eliminare ridondanze informative e garantire l'integrità referenziale tra tutte le entità del dominio applicativo.



7.2 Definizione DDL delle Tabelle, Chiavi Primarie ed Esterne

La struttura del database è definita attraverso i seguenti comandi DDL per la creazione delle entità principali:

- **Tabella Utenti:** memorizza le informazioni anagrafiche e i dati di autenticazione degli utenti (sia clienti che ristoratori).
 - id_utente (SERIAL, PRIMARY KEY): identificativo univoco progressivo generato automaticamente dal DBMS.
 - nome, cognome (VARCHAR(100), NOT NULL): anagrafica dell'utente.

- username (VARCHAR(100), UNIQUE, NOT NULL): identificatore per il login; il vincolo UNIQUE impedisce a livello relazionale la registrazione concorrente di nickname duplicati.
 - password (VARCHAR(255), NOT NULL): memorizza la chiave di autenticazione preventivamente cifrata con algoritmo AES.
 - data_nascita (DATE): data di nascita facoltativa.
 - domicilio (VARCHAR(150), NOT NULL): comune o luogo di residenza dell'utente.
 - ruolo (VARCHAR(20), NOT NULL): ruolo dell'account, vincolato dal controllo di dominio CHECK (ruolo IN ('CLIENTE', 'RISTORATORE')).
- **Tabella RistorantiTheKnife:** cataloga tutti i locali monitorati dalla piattaforma e ne descrive le caratteristiche geografiche e operative.
 - id_ristorante (SERIAL, PRIMARY KEY): chiave primaria univoca del ristorante.
 - nome (VARCHAR(150), NOT NULL), indirizzo (VARCHAR(200), NOT NULL), città (VARCHAR(100), NOT NULL), nazione (VARCHAR(100), NOT NULL): anagrafica e localizzazione geografica.
 - latitudine, longitudine (DECIMAL(9,6)): coordinate geografiche WGS84 a 6 cifre decimali (precisione metrica) impiegate per il calcolo della prossimità spaziale tramite formula di Haversine.
 - prezzo (DECIMAL(10,2)): prezzo medio indicativo per pasto in euro.

- consegna (BOOLEAN, DEFAULT FALSE): disponibilità del servizio di asporto/delivery.
- pren_online (BOOLEAN, DEFAULT FALSE): disponibilità della prenotazione online dei tavoli.
- **Tabella TipologiaCucina:** dizionario per la catalogazione dei generi gastronomici offerti.
 - id_tipo_cucina (SERIAL, PRIMARY KEY): identificativo univoco della cucina.
 - nome_cucina (VARCHAR(50), UNIQUE, NOT NULL): denominazione normalizzata della tipologia culinaria (es. ITALIAN, JAPANESE, VEGETARIAN).
- **Tabella Recensioni:** gestisce sia le recensioni principali rilasciate dai clienti, sia le repliche inviate dai ristoratori proprietari.
 - id_recensione (SERIAL, PRIMARY KEY): chiave primaria della recensione.
 - fk_id_ristorante (INT, NOT NULL, FK): riferimento a RistorantiTheKnife(id_ristorante) con clausola ON DELETE CASCADE.
 - fk_id_utente (INT, NOT NULL, FK): riferimento all'autore in Utenti(id_utente) con clausola ON DELETE CASCADE.
 - id_recensione_padre (INT, FK): chiave esterna ricorsiva su Recensioni(id_recensione) con clausola ON DELETE CASCADE. Assume valore -1 o NULL per le recensioni primarie; per le risposte assume l'ID della recensione a cui si risponde.
 - voto (INT): valutazione numerica vincolata dal controllo CHECK (voto >= 1 AND voto <= 5) per le recensioni primarie (impostato a -1 per le risposte).
 - commento (TEXT): testo descrittivo del feedback o della replica.
 - data (DATE, DEFAULT CURRENT_DATE): data di registrazione del record.

7.3 Tabelle Ponte e Normalizzazione delle Relazioni Multi-a-Molti

Per disaccoppiare e modellare correttamente le relazioni N:M tra le entità, sono state introdotte tre tabelle ponte con chiavi primarie composte:

- **Ristorante_Cucina:** associa ciascun ristorante a una o più tipologie culinarie.
 - `fk_id_ristorante` (INT, FK verso `RistorantiTheKnife`), `fk_id_tipo_cucina` (INT, FK verso `TipologiaCucina`).
 - Chiave primaria composta: PRIMARY KEY (`fk_id_ristorante`, `fk_id_tipo_cucina`).
- **Preferiti:** memorizza l'elenco dei ristoranti contrassegnati come preferiti da ciascun cliente.
 - `fk_id_utente` (INT, FK verso `Utenti`), `fk_id_ristorante` (INT, FK verso `RistorantiTheKnife`).
 - Chiave primaria composta: PRIMARY KEY (`fk_id_utente`, `fk_id_ristorante`) (previene duplicati per la stessa coppia utente-ristorante).
- **Gestione_Ristoranti:** associa ciascun ristoratore ai locali di cui detiene la proprietà e i permessi di amministrazione.
 - `fk_id_utente` (INT, FK verso `Utenti`), `fk_id_ristorante` (INT, FK verso `RistorantiTheKnife`).
 - Chiave primaria composta: PRIMARY KEY (`fk_id_utente`, `fk_id_ristorante`).

7.4 Integrità Referenziale, Vincoli ACID e Controllo delle Risposte

La consistenza della base di dati in ambiente concorrente e multi-client è presidiata tramite i seguenti meccanismi relazionali e transazionali:

- **Integrità Referenziale con ON DELETE CASCADE:** Tutte le foreign key che collegano tabelle figlie (Recensioni, Preferiti, Gestione_Ristoranti, Ristorante_Cucina) integrano la clausola di cancellazione a cascata. L'eventuale eliminazione di un ristorante o di un utente rimuove atomicamente tutti i record dipendenti senza generare tuple orfane nel database.
- **Vincolo Atomico "Al massimo una risposta per recensione":** Oltre ai controlli preventivi nel layer DAO, a livello DBMS il vincolo di business che autorizza una singola replica del ristoratore per ciascun feedback è blindato tramite un indice unico parziale su PostgreSQL:

SQL

```
CREATE UNIQUE INDEX idx_unica_risposta_per_recensione  
ON Recensioni (id_recensione_padre)  
WHERE id_recensione_padre IS NOT NULL AND id_recensione_padre <> -1;
```

Questo vincolo assicura l'impossibilità fisica di inserire risposte duplicate anche in caso di richieste concorrenti simultanee sul medesimo record padre.

- **Isolamento Transazionale:** Tutte le operazioni complesse che coinvolgono modifiche o cancellazioni multiple operano sotto transazioni esplicite (BEGIN TRANSACTION ... COMMIT / ROLLBACK), preservando le proprietà di Atomicità, Consistenza, Isolamento e Durabilità (ACID).